

Evolutionary Computation Lab 2

Maximum Satisfiability (MAXSAT)

Exercises 4 & 5

mpuneet063

May 2026

Contents

1	Pseudo-code Description of the Evolutionary Algorithm	2
1.1	Overview	2
1.2	Pseudo-code	2
2	Parameter Sensitivity Study	3
2.1	Experimental Setup	3
2.2	Step 1: Effect of Population Size (M)	4
2.3	Step 2: Effect of Mutation Rate (p_m)	5
2.4	Step 3: Effect of Tournament Size (k)	6
2.5	Summary of Findings	7

1 Pseudo-code Description of the Evolutionary Algorithm

1.1 Overview

The algorithm implemented for Exercise 3 is a generational Evolutionary Algorithm (EA) with elitism, uniform crossover, bit-flip mutation, and k -tournament selection. Each individual is a bitstring $x \in \{0, 1\}^n$, and fitness is the number of MAXSAT clauses satisfied.

- **Representation:** Bitstring $x \in \{0, 1\}^n$, where n is the number of Boolean variables.
- **Fitness:** Number of satisfied clauses (maximised).
- **Initialisation:** Each bit is drawn independently and uniformly from $\{0, 1\}$.
- **Selection:** k -tournament selection (with replacement).
- **Recombination:** Uniform crossover. Each gene is inherited from parent 1 with probability 0.5; otherwise, it is inherited from parent 2.
- **Mutation:** Independent bit-flip with per-bit probability p_m .
- **Replacement:** Generational with elitism: the global best individual is always preserved.
- **Termination:** Wall-clock time budget T (seconds).

1.2 Pseudo-code

Algorithm 1 Evolutionary Algorithm for MAXSAT

Require: Clauses C , Time Budget T , Population Size N , Mutation Rate p_m , Tournament Size k

Ensure: Best assignment x^*

```
1:  $P \leftarrow \text{InitializePopulation}(N)$ 
2:  $x^* \leftarrow \text{GetBest}(P, C)$ 
3: while  $\text{ElapsedTime}() < T$  do
4:    $P' \leftarrow \{x^*\}$  ▷ Elitism: carry over best individual
5:   while  $|P'| < N$  do
6:      $p_1 \leftarrow \text{TournamentSelection}(P, k)$ 
7:      $p_2 \leftarrow \text{TournamentSelection}(P, k)$ 
8:      $c \leftarrow \text{UniformCrossover}(p_1, p_2)$ 
9:      $c \leftarrow \text{BitFlipMutation}(c, p_m)$ 
10:    if  $\text{Fitness}(c, C) > \text{Fitness}(x^*, C)$  then
11:       $x^* \leftarrow c$ 
12:    end if
13:     $P' \leftarrow P' \cup \{c\}$ 
14:  end while
15:   $P \leftarrow P'$ 
16: end while
17: return  $x^*$ 
```

2 Parameter Sensitivity Study

2.1 Experimental Setup

The three parameters investigated are:

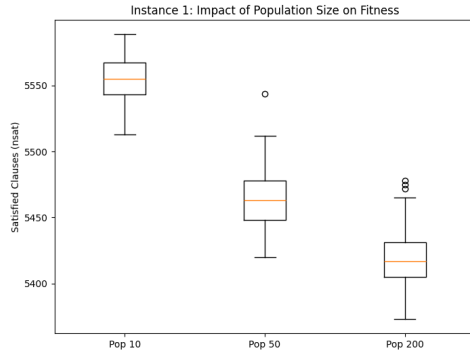
1. **Population Size** (M) $\in \{10, 50, 200\}$
2. **Mutation Rate** (p_m) $\in \{0.001, 0.05, 0.2\}$
3. **Tournament Size** (k) $\in \{2, 5, 10\}$

Experiments were conducted on three benchmark instances from the MSE17 Complete Un-weighted Benchmarks:

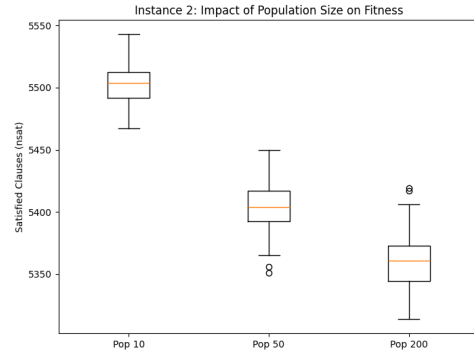
Instance	Filename	Description
I1	frb20-11-1.partial.wcnf	220 variables, 5833 clauses
I2	frb20-11-2.partial.wcnf	220 variables, 5794 clauses
I3	frb20-11-3.partial copy 2.wcnf	220 variables, 5854 clauses

For each configuration, 100 independent repetitions were run within a fixed wall-clock budget of 2 seconds. A one-factor-at-a-time (OFAT) protocol was followed relative to the baseline configuration of $M = 50$, $p_m = 0.05$, and $k = 2$. Results are shown as boxplots.

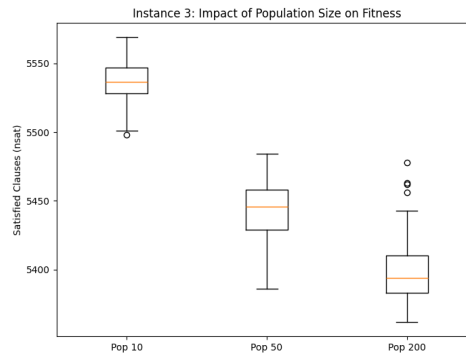
2.2 Step 1: Effect of Population Size (M)



(a) Instance 1



(b) Instance 2



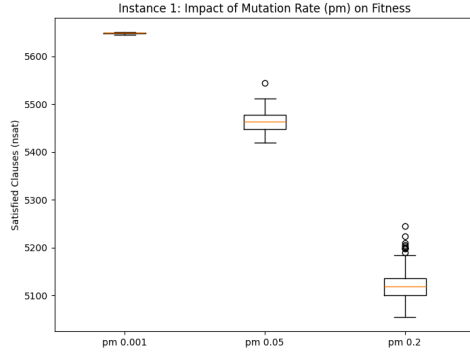
(c) Instance 3

Figure 1: Effect of population size on fitness. Fixed: $p_m = 0.05$, $k = 2$.

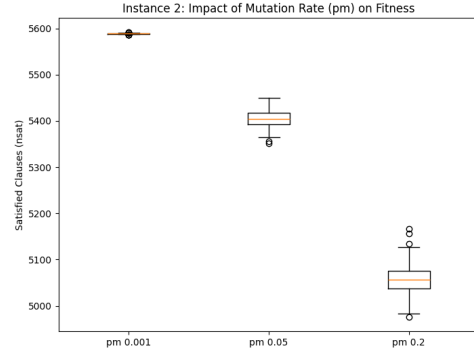
Observations: A clear monotonic decrease in median fitness is observed as population size increases across all instances. A population of 10 consistently yields the highest median fitness, while a population of 200 yields the lowest.

Interpretation: Under a strict wall-clock time budget, generating and evaluating a larger population consumes more time per generation. Consequently, a population of 200 completes far fewer generations than a population of 10. The reduced number of selection and recombination cycles prevents the larger population from converging to higher fitness levels within the allocated time.

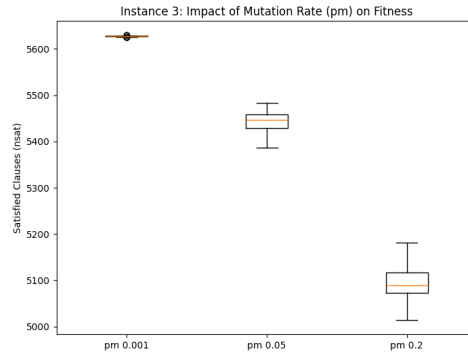
2.3 Step 2: Effect of Mutation Rate (p_m)



(a) Instance 1



(b) Instance 2



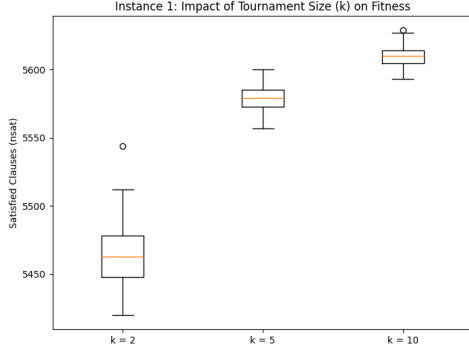
(c) Instance 3

Figure 2: Effect of mutation rate on fitness. Fixed: $M = 50, k = 2$.

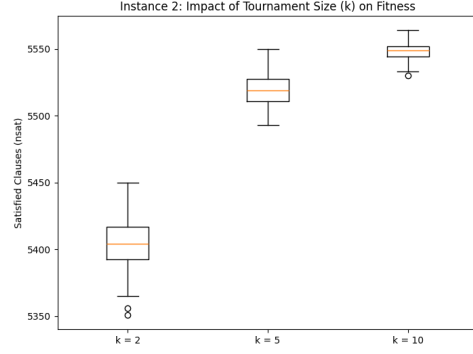
Observations: Median fitness degrades severely as the mutation rate increases from 0.001 to 0.2 across all instances. The lowest mutation rate tested (0.001) significantly outperforms the baseline (0.05) and the high rate (0.2).

Interpretation: High mutation rates disrupt the beneficial genetic material accumulated through crossover and selection. At $p_m = 0.2$, the algorithm approximates a random search, failing to exploit good partial solutions. The theoretical default mutation rate of $1/n$ (approx. 0.0045 for $n = 220$) aligns with the superior performance observed at $p_m = 0.001$.

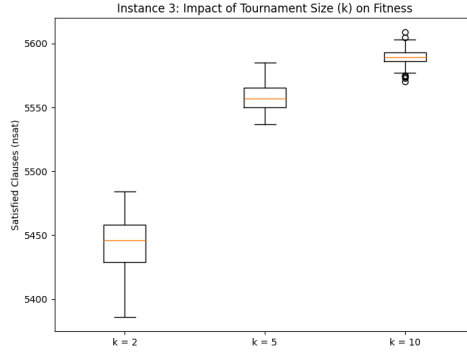
2.4 Step 3: Effect of Tournament Size (k)



(a) Instance 1



(b) Instance 2



(c) Instance 3

Figure 3: Effect of tournament size on fitness. Fixed: $M = 50, p_m = 0.05$.

Observations: Increasing tournament size from $k = 2$ to $k = 5$ and $k = 10$ results in a marked increase in median fitness across all instances. The variance also decreases slightly at higher selection pressures.

Interpretation: Higher tournament sizes increase selection pressure, driving the population rapidly toward high-fitness regions. The baseline setting of $k = 2$ provides insufficient selection pressure, allowing weaker individuals to survive and slowing overall convergence. While higher values improve performance here, excessively high selection pressure combined with small populations may eventually lead to premature convergence, though that threshold was not clearly breached at $k = 10$.

2.5 Summary of Findings

Parameter	Values Tested	Best	Key Finding
Population Size (M)	10, 50, 200	10	Smaller populations complete more generations within the fixed time budget, consistently yielding higher median fitness.
Mutation Rate (p_m)	0.001, 0.05, 0.2	0.001	High mutation rates destroy beneficial partial solutions faster than recombination can build them. Lower rates preserve exploitation.
Tournament Size (k)	2, 5, 10	10	Increased selection pressure accelerates convergence to high-fitness regions, outperforming the weak pressure of $k = 2$.

Overall Recommended Configuration: $M = 10$, $p_m = 0.001$, $k = 10$.

This configuration maximizes generation turnover within the time limit (small M), preserves good solutions via conservative mutation (p_m), and employs strong selection pressure to drive convergence (k).